בס"ד

NodeJS / Request Pipeline – Exercise 5

Create an Express app that supports login and role-based access control.

JWT – Introduction

JWT (JSON Web Token) is a compact and secure method for representing a user's identity between a client and a server.
When a user logs in successfully, the server creates a token that contains some information about the user (for example: user ID and role). The token is encrypted using a secret key. The token is then sent to the client, which stores it (usually in local storage or a cookie) and includes it in every subsequent request to verify their identity.
On every request, the server takes this token and validates it using the secret key to make sure it was generated by it. The content of the token is returned from the verification function, allowing the server to check the user's permissions (such as their role).
In this task, we'll use the "jsonwebtoken" npm package to sign tokens and validate them.

Requirements

- Create a simple server that handles two resources, each on a separate router:
    - **Users** – for managing user data.
    - **Items** (use an appropriate name) – any other resource of your choice (e.g. products, books, posts, etc.).
- Use an **array** as your database for both users and items.

Functional Requirements

1. **Auth utils**
    - Create a folder named "common". Add a file with the name "auth-utils.ts".
    - Create a secret key that will be used to sign the tokens (can be any random string). Store it in a constant variable.
    - Add a class with the following static methods:
        - signJwt(payload) - Returns a JWT that is signed by the secret key. Use the "jwt.sign" function from the "jsonwebtoken" package.
        - validateJwt(token) – Validates the JWT using the secret key, and returns the json data. Use the "jwt.verify" function from the "jsonwebtoken" package.
2. **Users router** - /api/users
    - The user interface includes:
        - Username (The unique identifier)
        - Password
        - Role (Can be "admin" / "user")
    - Support the following endpoints:
        - POST / – add a new user, add validation on the role field.
        - POST /login:

1. Gets a username and password in the request body.
2. Checks if the user exists and the password is correct, if not – return 401 status code.
3. Create a JWT token that contains the username & role using the auth utils class.
4. Return the token to the user.

3. **Items router** - /api/[items]
    - Support the following endpoints:
        - POST / - add a new item, no need to validate the input.
        - GET / - return all items in the list.
4. **Authentication Middleware**

   Create a middleware that:
    - Reads the token from the Authorization header:
        - The header name: **Authorization**
        - The header value: **Bearer <token>.** (Split the string by the space, and take the second part of the result).
    - Verifies the token using the auth utils class.
    - If valid, adds the decoded user information to req.user.
    - If missing or invalid, returns 401 Unauthorized.
5. **Authorization Middleware**

   Create a middleware that:
    - checks the user's role (from req.user.role).
    - It should accept an array of allowed roles.
    - If the current user's role is not allowed, return 403 Forbidden.
6. **Use the middlewares**
    - The authentication middleware should run for all routers, at the app level.
    - The authorization middleware should be added as follows:
        - Adding users – only admin
        - Login – admin & user
        - Items endpoints – admin & user

Notes

- Run the app, test all routes from Postman, and submit the Postman collection file with the project.
- Make sure to write clean code that follows all conventions.